

Bianca Ackermann - Unsplash

Regex
(puzzle time!)

Adapted from materials from Dr. Carrier

Recall…

Recall…

Questions:

How many pokemon are Grass type?

Recall…

Questions:

How many pokemon are Grass type?

How many pokemon are Grass *and* Psychic type?

Recall…

Questions:

How many pokemon are Grass type?

How many pokemon are Grass *and* Psychic type?

How many pokemon are Grass *or* Psychic type?

Pulling from other classes

Questions:

- How do you define the set of all integers?

Pulling from other classes

Questions:

- How do you define the set of all integers?
- How do you define the set of all **even** integers?

Advanced searching

Advanced searching

Regular expression (regex)

Advanced searching

Regular expression (regex)

Defines a ***set*** of strings

Advanced searching

Regular expression (regex)

- Defines a ***set*** of strings
- Basic strings can represent themselves

Advanced searching

Regular expression (regex)

- Defines a ***set*** of strings
- Basic strings can represent themselves
  - e.g., dog is just the string dog

Advanced searching

Regular expression (regex)

- Defines a ***set*** of strings
- Basic strings can represent themselves
  - e.g., dog is just the string dog
- However, we also have *special* characters

Advanced searching

Regular expression (regex)

- Defines a ***set*** of strings
- Basic strings can represent themselves
  - e.g., dog is just the string dog
- However, we also have *special* characters
  - These allow us to match multiple strings

The basics

a matches the string “a”

The basics

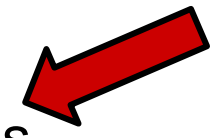
a matches the string “a”

a+ matches one or more “a”s

The basics

a matches the string “a”

a+ matches one or more “a”s



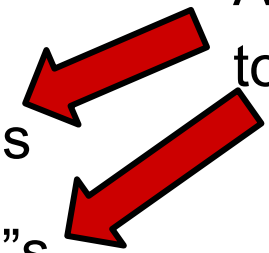
Applies to item directly to the left of the +

The basics

a matches the string “a”

a+ matches one or more “a”s

a* matches zero or more “a”s



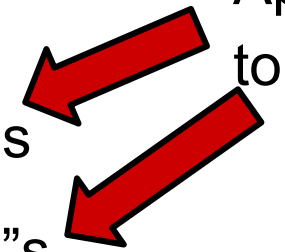
Applies to item directly to the left of the + or *

The basics

`a` matches the string “a”

`a+` matches one or more “a”s

`a*` matches zero or more “a”s



Applies to item directly to the left of the + or *

What does `lo+l` match?

The basics

`a` matches the string “a”

`a+` matches one or more “a”s

`a*` matches zero or more “a”s

What does `lo+l` match?

Where does this differ from `lo*l`?

More basics

What does `(ab)+` match?

More basics

What does `(ab)+` match?

Parentheses can group characters (called a capture group)

More basics

What does `(ab)+` match?

Parentheses can group characters (called a capture group)

Which ones of these wouldn't match? Why?

“ab” “abab” “” “ba” “abababababab” “aba” “aab”

A common character

What does this command do?

```
cat file_*
```

A common character

What does this command do?

```
cat file_*
```

What if we want that wildcard functionality in regex?

A common character

What does this command do?

```
cat file_*
```

What if we want that wildcard functionality in regex?

. (a dot/period) - matches *any* character

A common character

What does this command do?

```
cat file_*
```

What if we want that wildcard functionality in regex?

. (a dot/period) - matches *any* character

How do we then match the same strings as the command above?

A common character

What does this command do?

```
cat file_*
```

What if we want that wildcard functionality in regex?

. (a dot/period) - matches *any* character

How do we then match the same strings as the command above?

```
file_.*
```

Applying restrictions

Example: what would `(b.d)+` match? `b.+d`?

Applying restrictions

Example: what would `(b.d)+` match? `b.+d`?

What if we want to restrict the wildcard to only match vowels?

Applying restrictions

Example: what would `(b.d)+` match? `b.+d`?

What if we want to restrict the wildcard to only match vowels?

We use character classes `[ ]` - take ONE of these chars

Applying restrictions

Example: what would `(b.d)+` match? `b.+d`?

What if we want to restrict the wildcard to only match vowels?

We use character classes `[ ]` - take ONE of these chars

`b[aeiou]d`

Applying restrictions

Example: what would `(b.d)+` match? `b.+d`?

What if we want to restrict the wildcard to only match vowels?

We use character classes `[ ]` - take ONE of these chars

`b[aeiou]d`

How do `(ab)+` and `[ab]+` differ?

Applying restrictions

Example: what would `(b.d)+` match? `b.+d`?

What if we want to restrict the wildcard to only match vowels?

We use character classes `[ ]` - take ONE of these chars

Note we can also use tr-like character classes:

```
[[:digit:]] [a-z]
```

Applying restrictions

Example: what would `(b.d)+` match? `b.+d`?

What if we want to restrict the wildcard to only match vowels?

We use character classes `[ ]` - take ONE of these chars

Note we can also use tr-like character classes:

```
[[:digit:]] [a-z]
```

Two sets of [ ] when using tr-like classes

Applying restrictions

Example: what would `(b.d)+` match? `b.+d`?

What if we want to restrict the wildcard to only match vowels?

We use character classes `[ ]` - take ONE of these chars

Note we can also use tr-like character classes:

```
[[:digit:]] [a-z]
```

Two sets of `[]` when using tr-like classes
Outer: This is a character class
Inner + colons: We are use a tr-style set

Applying restrictions

Example: what would `(b.d)+` match? `b.+d`?

What if we want to restrict the wildcard to only match vowels?

We use character classes `[ ]` - take ONE of these chars

Note we can also use tr-like character classes:

`[[:digit:]] [a-z]`

We can also invert them: `[^[:digit:]] [^0]`

Applying restrictions

What would `[[:digit:]][[:digit:]][[:digit:]]` match?

What would `[^a-z][a-z]*` match?

Matching the forbidden characters

What does `foo.txt` match?

Matching the forbidden characters

What does `foo.txt` match?

What if we want it to only match “foo.txt”?

Matching the forbidden characters

What does `foo.txt` match?

What if we want it to only match “foo.txt”?

```
foo\.txt
```

Recall…

How many pokemon are Steel *or* Psychic type?

Recall…

How many pokemon are Steel *or* Psychic type?

```
Steel|Psychic
```

What does this match?

```
a dog|cat
```

What does this match?

```
a dog|cat
```

Matches “a dog” or “cat”. Does not match “a cat”

What does this match?

```
a dog|cat
```

Matches “a dog” or “cat”. Does not match “a cat”
The | operator is greedy. Bound it with parens:

```
a (dog|cat)
```

More counts

`a` matches the string “a”

`a+` matches one or more “a”s

`a*` matches zero or more “a”s

More counts

a	matches the string “a”
a+	matches one or more “a”s
a*	matches zero or more “a”s
a?	matches zero or one “a”s

More counts

`a` matches the string “a”

`a+` matches one or more “a”s

`a*` matches zero or more “a”s

`a?` matches zero or one “a”s

`a{n}` matches n “a”s

More counts

`a` matches the string “a”

`a+` matches one or more “a”s

`a*` matches zero or more “a”s

`a?` matches zero or one “a”s

`a{n}` matches n “a”s

`a{n,m}` matches between n and m “a”s

Final characters

^a matches “a”s at the beginning of the line

a$ matches “a”s at the end of the line

Some examples

What do these match?

Some examples

What do these match?

```
The (dog|cat) ra+n away$
```

Some examples

What do these match?

```
The (dog|cat) ra+n away$
^bee+s*
```

Some examples

What do these match?

```
The (dog|cat) ra+n away$
^bee+s*
[Ll][ol]{2}[ol]*
```

Some examples

Create a regex to:

Match “ab” “aba” “abb” “abba” “abab” “abbb” “abaa”
and nothing else

Order of operations

	ERE Precedence (from high to low)	
1	Collation-related bracket symbols	[==] [::] [..]
2	Escaped characters	\<special character>
3	Bracket expression	[]
4	Grouping	()
5	Single-character-ERE duplication	* + ? {m,n}
6	Concatenation	
7	Anchoring	^ $
8	Alternation	|

Table from POSIX docs, formatting from StackOverflow

Using regular expressions

We are using extended regular expressions (ERE)

Using regular expressions

We are using extended regular expressions (ERE)

You can use them with `grep` with `-E`:

```
grep -E “ab[ab]{0,2}” file.txt
```

Grep actually stands for “global regular expression print”

Using regular expressions

We are using extended regular expressions (ERE)

You can use them with `grep` with `-E`:

```
grep -E “ab[ab]{0,2}” file.txt
```

Grep actually stands for “global regular expression print”

Grep returns any lines with a match
To return just the match, add a `-o`

Using regular expressions

We are using extended regular expressions (ERE)

You can use them with `grep` with `-E`:

```
grep -E “ab[ab]{0,2}” file.txt
```

Grep actually stands for “global regular expression print”

Grep returns any lines with a match
To return just the match, add a `-o`

Note that other commands use /pattern/ to denote regex!

Practice questions

1. Which strings match the regex: `[^5][[:digit:]]+`
   “12”, “3”, “50”, “15”, “b0”, “10000”, “1050”, “$10”, “$4.50”, “2!”

2. Create a regex to:
   a. Look for CIS course codes (bonus: grad vs undergrad)
   b. Match email addresses

3. What does this match?

```
[[:digit:]]{10}|\([[:digit:]]{3}\)[[:digit:]]{3}-[[:digit:]]{4}
```